

Ejercicio 1

Teniendo en cuenta que actualmente en España se aplican tres tipos diferentes de IVA:

- IVA general, que actualmente es del 18%
- IVA reducido, que actualmente es del 8%
- IVA superreducido, que actualmente es del 4%

Se pide que diseñéis e implementéis la jerarquía de clases necesarias para representar los diferentes tipos de productos que se venden en un supermercado según el tipo de IVA. De todos los productos se desea mantener un código que los identifica, su descripción y su precio sin IVA. De todas las clases de productos se ha de poder consultar el código, consultar y modificar la descripción, consultar y modificar el precio sin IVA, y consultar el precio con IVA. El resultado de la consulta del precio con IVA ha de ser el correcto según el tipo de producto.

Ejercicio 2

Una compañía aérea necesita mantener información de los aeropuertos con los que opera y de la duración en minutos de los vuelos que tiene entre los diferentes aeropuertos, en el caso de no tener vuelo quiere mantener como duración 0 minutos. La compañía quiere poder inicializar el sistema únicamente con los nombres de todos los aeropuertos con los que opera, para después ir actualizado los vuelos y realizando consultas sobre ellos dinámicamente. Concretamente quiere poder actualizar la duración del vuelo entre dos de los aeropuertos con los que opera, poder consultar si hay vuelo entre dos aeropuertos y poder consultar la duración del vuelo entre dos aeropuertos. Se pide diseñar la clase `CompañiaAerea` de manera que:

- Almacene el nombre de los aeropuertos en una tabla unidimensional llamada `nombreAeropuertos`
- Almacene la información de la duración de los vuelos en una tabla bidimensional `duracionVuelos` de manera que el valor de la posición `[i,j]` sea la duración del vuelo entre el aeropuerto `nombreAeropuertos[i]` y el aeropuerto `nombreAeropuertos[j]`
- Tenga un método que dados el nombre del aeropuerto origen, el nombre del aeropuerto destino y la duración del vuelo (en caso de que opere con el aeropuerto origen y con el aeropuerto destino) actualice la información de la duración del vuelo entre los dos aeropuertos.
- Tenga un método para consultar la duración del vuelo entre dos aeropuertos (origen, y destino). Este método ha de devolver la duración actual del vuelo entre los dos aeropuertos, en caso de que no haya vuelo ha de devolver un 0.
- Tenga un método para consultar si hay un vuelo directo entre dos aeropuertos (origen y destino). Este método ha de devolver un valor cierto si hay un vuelo entre los dos aeropuertos y falso en caso contrario.

Ejercicio 3

Un polideportivo quiere una aplicación para gestionar las cuotas mensuales de sus socios. El polideportivo tiene tres tipos de socios: socios que se pueden inscribir a un máximo de 5 actividades semanales, socios que se pueden inscribir a un máximo de 10 actividades semanales y socios abiertos que pueden realizar todas las actividades semanales que quieran sin inscripción previa. Todos los socios tienen un DNI que los identifica, el nombre y apellidos, y una cuota mensual base de 10.0 euros. Todos los socios por actividades tienen además una cuota base mensual de 2.0 euros por actividad a la que están inscritos, los que se pueden

inscribir a un máximo de 5 actividades tienen además un coste adicional por actividad de 2.0 euros mientras que los que pueden inscribirse a un máximo de 10 tienen un coste adicional mensual de 6.0 euros y un coste adicional por actividad de 1 euro. Finalmente los socios abiertos tienen un coste adicional mensual de 38.0 euros.

Diseñar las clases Socio, SocioAbierto, SocioMax5Actividades y SocioMax10Actividades de manera que implementen la jerarquía de clases correspondiente a los socios del polideportivo, el método cuotaMensual() calcule correctamente la cuota mensual de los diferentes socios y el método toString() escriba el tipo de socio, el número de actividades a la que está inscrito el socio en caso que sea un socio por actividades, el DNI y el nombre y apellidos. Se adjunta la clase PruebaSocios para que podáis probar el funcionamiento de las clases. Si el funcionamiento es correcto el programa producirá una salida con los siguientes datos (puede variar el formato dependiendo de vuestra implementación del método toString()):

Socio maximo 5 actividades
Numero de actividades: 2
DNI: 45778967A
Nombre y Apellidos: Iker Garrido
Cuota mensual: 18.0

Socio maximo 5 actividades
Numero de actividades: 5
DNI: 34567896E
Nombre y Apellidos: Juan Lopez
Cuota mensual: 30.0

Socio maximo 10 actividades
Numero de actividades: 5
DNI: 56738965F
Nombre y Apellidos: Pepe Martin
Cuota mensual: 31.0

Socio maximo 10 actividades
Numero de actividades: 6
DNI: 40896552C
Nombre y Apellidos: Maria Pompas
Cuota mensual: 34.0

Socio maximo 10 actividades
Numero de actividades: 10
DNI: 41654329H
Nombre y Apellidos: Isabel Perez
Cuota mensual: 46.0

Socio Abierto
DNI: 43456796D
Nombre y Apellidos: Ariadna Marin
Cuota mensual: 48.0

Ejercicio 4

Diseñar la clase Polinomio de manera que la clase PruebaPolinomio funcione correctamente. La clase Polinomio ha de representar un polinomio como una tabla de n coeficientes reales

donde n es igual al grado del polinomio más 1 y donde el elemento i de la tabla es el coeficiente de grado i del polinomio (que puede ser 0). El método suma ha de devolver el polinomio resultante de sumar el polinomio al que se le aplica el método con otro polinomio, que no tiene porque ser del mismo grado. Igualmente el método resta ha de devolver el resultado de la resta. El constructor Polinomio() ha de crear un polinomio de grado 0 con coeficiente 0. El constructor Polinomio(double[] coeficientes) ha de crear un polinomio de grado igual al grado más alto con coeficiente diferente de 0.0 con los coeficientes que hay en la tabla.

Se adjunta la clase PruebaPolinomio para que podáis probar el funcionamiento de la clase Polinomio. Si el funcionamiento es correcto el programa producirá una salida con los siguientes datos:

p1: $-0.3x^3+5.1x^2+0.0x+2.3$

p2: $-0.3x^3+5.1x^2-1.2x+2.3$

p1.suma(p1): $-0.6x^3+10.2x^2+0.0x+4.6$

p2.suma(p1): $-0.6x^3+10.2x^2-1.2x+4.6$

p1.resta(p2): $1.2x+0.0$

p3: $-2.13x^5+1.67x^4-2.4x^3+3.1x^2-2.5x+4.5$

p4: $-10.5x^3+4.5x^2-2.0x+3.0$

p3.suma(p4): $-2.13x^5+1.67x^4-12.9x^3+7.6x^2-4.5x+7.5$

p4.resta(p3): $2.13x^5-1.67x^4-8.1x^3+1.4x^2+0.5x-1.5$